

CNN Emotion Detection Final Report

Spencer Scherger & Griffin Kuchar

May 1st, 2026

1 Introduction and Problem Description

Facial emotion detection is one of the most complex problems at the intersection of computer vision and machine learning, particularly when subject to live video systems and under performance constraints. The project addresses this problem through a mapping of a video stream of a user's face to an emoji that best represents their current emotion. The problems we face are creating an efficient live data pipeline, training an accurate convolutional neural network (CNN) under performance constraints, and maintaining low latency and scalability of an interactive application. To solve this, we explored the trade-offs of implementing data pipeline algorithms and machine learning algorithms on a CPU versus a GPU through benchmarking their run times. The end result is an optimal combination of algorithms that deliver the best performance to the user.

2 Dataset and Workload Description

From kaggle.com, we found two different, free to use, facial emotion datasets:

- <https://www.kaggle.com/datasets/msambare/fer2013>
- <https://www.kaggle.com/datasets/mh0386/facial-emotion>

We collated these two sets into a complete dataset (split into train/test) that included seven different emotion labels: angry, disgust, happy, fear, sad, surprised, and neutral. To simplify the decision boundary for our model, we reduced the model's prediction set to three emotion classes: positive, neutral, and negative. This reduction required mapping the seven given emotion folders to a custom three emotion dataset. The custom mapping was designed as follows:

- happy $\rightarrow$ positive
- surprised, neutral $\rightarrow$ neutral
- angry, fear, disgust, sad $\rightarrow$ negative

The complete dataset breakdown is as follows:

Emotion Class	Train	Test	Total
Positive (happy)	7,229	1,779	9,008
Neutral (surprised, neutral)	8,178	2,080	10,258
Negative (angry, disgust, fear, sad)	13,414	3,359	16,773
Total	28,821	7,218	36,039

Table 1: Dataset split by emotion class.

We then created a preprocessing Python notebook to square, grayscale, and normalize each image in order to prepare the images to match the model’s expected format. Preprocessing specifics are as follows:

- square = resize each image into a fixed 48×48 pixel structure
- grayscale = 8 bit standard Luma grayscaleing per pixel:

$$\text{new pixel color} = (0.299 \times R) + (0.587 \times G) + (0.114 \times B)$$

- normalize = per-image min-max normalization into the range $[-1, 1]$ per pixel:

$$\text{new pixel value} = 2 \cdot \frac{(\text{value} - \text{min})}{\text{max} - \text{min}} - 1$$

3 CPU Baseline Implementation

The frontend was implemented using Streamlit and utilizes OpenCV to take a frame-by-frame live video input feed of one person’s face, and passes every n th frame, where n is a configurable parameter, into a custom CNN object. The instance of the CNN makes an inference and returns the predicted emotion back into the frontend where the emoji is then displayed.

The CNN uses two convolutional layers to extract visual features from the face. The first layer applies 16 learned 3×3 filters to detect small patterns and edges, while the second layer applies 32 learned 3×3 filters to combine patterns into more complex and meaningful features. After each convolution, a ReLU activation function keeps the positive features while removing negative ones, which adds nonlinearity so the network can model more complexities. Then each convolution block is followed by a 2×2 max pooling, reducing image dimensions and preserving the strongest features, which serves to reduce computation cost. Finally, the last pooled feature map is flattened into a one-dimensional vector and passed through a fully connected hidden layer of 128 units, followed by dropout to reduce overfitting. The final fully connected layer produces three logits which get converted to probabilities using softmax during prediction. Training also employs cross-entropy loss, L2 regularization, backpropagation, and a simple

stochastic gradient descent. Once the model is trained, the weights are saved to a `.npz` file and loaded by the frontend application.

4 GPU Design and Implementation

The primary speedup from utilizing a GPU for our project is achieved through model training. A GPU drastically improves performance by computing the matrix operations in parallel across thousands of threads, far exceeding what a CPU is able to achieve through vectorization across its cores. A secondary, less significant, speedup in our project is found by utilizing a GPU to implement an optimized SIMT-kernel that operates on each of the 48×48 pixels during the live feed image preprocessing procedure. Although in the CPU implementation the live image feed preprocessing procedure utilizes parallel operations implicitly through NumPy, the corresponding custom CUDA kernel version in the GPU implementation yields faster performance.

The CNN has been fully trained on the GPU using custom kernels written through CuPy 12's `RawKernel` interface. Training was performed on the NCSA Delta HPC cluster; the batch submission script is provided in `run.slurm`. Every core component of the forward and backward pass (convolution, ReLU, pooling, softmax, and cross entropy loss) has been reimplemented as a CUDA kernel. Additionally, the NumPy live feed image preprocess procedure has been replaced with an optimized CUDA kernel implementation. The finalized model weights are containerized in `emotion.cnn.weights.npz`, which can be loaded to the Streamlit frontend at runtime for model inference.

```
1 void relu_forward(const float* x, float* out, const int size) {
2     int i = blockIdx.x * blockDim.x + threadIdx.x;
3
4     if (i < size) {
5         float value = x[i];
6         out[i] = value > 0.0f ? value : 0.0f;
7     }
8 }
```

Listing 1: Example CUDA Kernel - ReLU Forward Pass

5 Experimental Methodology

The experimental methodology measured both real-time application performance and model training performance to evaluate practicality of different implementations. For the frontend benchmark, every method was evaluated while `app.py` was in execution. In other words, the benchmark focuses on whether or not the CPU implementation could maintain responsive live interaction while performing face detection. This benchmarking was done through Python's built-in `time` library, and specifically through calls to `time.perf_counter()`. This is

the standard for measuring short durations and uses the system’s highest resolution clock.

For the benchmarking of the model training process, the training script reports average batch loss and accuracy after every epoch. Furthermore, the best epoch is saved as a checkpoint and early stopping is supported to protect against overfitting, based on validation accuracy. The timing of the model training is also done with `perf_counter()` and is printed as `total_training_time`. This simplified approach allows for quick comparisons of how CPU and GPU implementations differ when looking at batch size, epoch settings, learning rates, etc.

6 Results and Performance Analysis

6.1 CPU vs GPU Results

Our project implementations can be evaluated across two primary metrics: time and accuracy. Specifically, the GPU implementation’s model showed improvement in both time to train and test accuracy over the CPU’s corresponding model. Additionally, the GPU implementation recorded time improvement over the CPU implementation in live feed image preprocessing. The complete performance breakdown is as follows:

6.1.1 Model Metrics

Training was performed on the full train set over 20 epochs using identical hyperparameters across both implementations, achieving a $9.07\times$ speedup on the GPU.

Metric	CPU	GPU
Training Time	86m 12s	9m 30s
Overall Train Accuracy	0.781	0.713
Train Accuracy (per class)	positive: 0.836 neutral: 0.676 negative: 0.807	positive: 0.821 neutral: 0.657 negative: 0.609
Train Confusion Matrix	$\begin{pmatrix} 6041 & 456 & 732 \\ 508 & 3374 & 1111 \\ 403 & 530 & 3911 \end{pmatrix}$	$\begin{pmatrix} 5932 & 609 & 688 \\ 534 & 3280 & 1179 \\ 573 & 1319 & 2952 \end{pmatrix}$
Overall Test Accuracy	0.648	0.694
Test Accuracy (per class)	positive: 0.750 neutral: 0.537 negative: 0.614	positive: 0.822 neutral: 0.649 negative: 0.555
Test Confusion Matrix	$\begin{pmatrix} 1335 & 166 & 278 \\ 161 & 667 & 415 \\ 190 & 293 & 768 \end{pmatrix}$	$\begin{pmatrix} 1463 & 144 & 172 \\ 131 & 807 & 305 \\ 174 & 383 & 694 \end{pmatrix}$

Table 2: CNN training and evaluation metrics: CPU vs GPU. Confusion matrix rows are true labels and columns are predicted labels, ordered (positive, neutral, negative).

6.1.2 Live Feed Preprocessing Metrics

Metric	CPU	GPU
Avg per-frame preprocessing time (ms)	0.066	0.041

Table 3: Live feed preprocessing benchmark, averaged over 50 consecutive frames.

The GPU implementation achieves a $\sim 1.61\times$ speedup over the CPU baseline.

6.2 Performance Analysis

6.2.1 CPU Implementation

The CPU implementation of the live feed image preprocessing is relatively strong. This is because the real-time workload is kept small by only processing every n th frame (with each frame only consisting of 48×48 pixels), and due to CNN inference calls being relatively lightweight since the model is small.

The CPU implementation of the CNN model, however, was extremely weak. Performance was not impressive, as the initial accuracy was well below similar CNN models utilizing the same dataset previously written and claimed by others publicly on the Kaggle dataset comments. This is because training requires more computation than inference. More specifically, on a CPU, each filter must

be applied sequentially across many positions and images, and the backward pass must also sequentially compute gradients, weights, and biases. Many of the operations are performed on vectors or matrices, meaning the time complexity of these algorithms is usually $O(n)$ or $O(n^2)$ respectively.

Although the training accuracy of the CPU CNN was greater than the GPU's, the true measurement is the test accuracy, in which the GPU proved to perform better. There is less difference in training/testing accuracy for the GPU than the CPU, indicating the GPU's model endured less overfitting on training data. Ultimately, the important metric a GPU should improve upon is time, which is the case for our project. However, it is interesting to analyze how each implementation differs in accuracy.

6.2.2 GPU Implementation

The speedup gained from the GPU implementation during live feed image preprocessing is rather insignificant. This is due to the overhead of launching a CUDA kernel on a GPU (transferring data and scheduling) being relatively much greater than the workload size of the preprocessing kernel. In a theoretical situation where our workload size was greater (higher resolution images) the kernel launching overhead would be less significant relative to the workload, and a greater speedup would be achieved.

However, with a speedup achieved of $9.07\times$, the GPU excelled at training our model. By parallelizing the matrix and vector mathematical operations in each convolutional layer (convolution, activation, pooling, loss calculation, and gradient computation) across thousands of CUDA threads rather than running them sequentially on a single CPU core, a massive reduction in model training time is achieved.

7 Discussion and Lessons Learned

Besides the expected takeaway that utilizing a GPU improves image processing speed and model training time, the biggest lesson we learned was that completely implementing a project on a CPU or completely on a GPU is not optimal; rather a robust project is able to identify which aspects should utilize SIMD kernels. The performance increase on parallel operations using a GPU is clear, however the true engineering insight comes from understanding when to not use a GPU, even for tasks that may be standard GPU workloads. For example, we navigated the tradeoff of using the CUDA kernel for live feed image preprocessing. Although image processing operations excel on a GPU, given the overhead of the kernel launch relative to the small 48×48 workload, the $1.61\times$ speedup did not justify the added complexity for end-user inference, and we made the decision to keep live feed image preprocessing a CPU task. By utilizing a GPU for model training and weight computation, and saving and containerizing the

weights for CPU live feed image preprocessing and inference, we were able to provide the best user experience.

8 Future Work

Although we are proud of the current implementation of our project, there are a few possible additions that could elevate it further. First, one natural direction is improving the performance of our model. This could look like experimenting with different machine learning or deep learning model architectures, or with different hyperparameters. Some theoretical models to test are SVM (Support Vector Machine) due to its ability to define non-linear decision boundaries with the kernel trick, or ViT (Vision Transformer) due to its strong performance on image classification benchmarks. The core mathematical operations of these model architectures can be CUDA kernel optimized similarly to a CNN. Secondly, hyperparameter tuning on our current model may improve performance as well. Experimenting with different image pixel counts, learning rates, CNN layer counts, convolution/pooling window sizes, and/or activation functions may improve prediction accuracy. Thirdly, locating and training the current CNN on a higher count of more precisely labeled images is a simple direction to improve accuracy.

Detaching from model improvements, an additional possible feature to be added in the future could be allowing multiple faces into the live image feed, and running emotion inference on all faces in parallel. This could be implemented as a multi-threaded CPU application in which each thread loads in pre-trained model weights and performs inference. The pre-trained weights would continue to be computed via GPU, as described in Section 4.

9 Team Contribution Statement

As a team, efforts were split into two main goals. Spencer focused on building the model and training script, while Griffin focused on the application side, including the frontend and backend that allowed the model to be used through the project interface.

The work was organized by dividing the project into clear responsibilities and using GitHub branches to work on different parts at the same time. After completing features on separate branches, the work was merged together and any integration issues were resolved as they appeared.

The collaboration relied on clear communication and planning. In person, we discussed how data needed to be formatted and what outputs each part of the project should expect. For example, Griffin explained that the input data would be 48x48 grayscale images before writing that part of the code, which allowed

Spencer to design the model around that assumption and continue working without waiting for a fully completed frontend implementation. This style of communication helped avoid mismatches between components and made it easier to combine our work into a single complete project.